Spring Security — Zero to Master

Complete Guide with Definitions, Concepts, and Code — JWT & OAuth2

Chapter 1: Getting Started

Spring Security: a framework that provides authentication, authorization, and protection against common attacks for Java applications, integrating with Spring/Spring Boot via a chain of servlet filters.

Authentication: the process of verifying who a user is (e.g. checking a username and password).

Authorization: the process of verifying what an authenticated user is allowed to do (e.g. checking roles/permissions before granting access to a resource).

1.1 Creating a Simple Spring Boot App Without Security

Before adding security, a plain Spring Boot REST controller exposes endpoints with no access restrictions at all — anyone can call /hello or /admin.

```
<!-- pom.xml dependencies -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello, World!";
    }

    @GetMapping("/admin")
    public String admin() {
        return "Admin Access Only!";
    }
}
```

1.2 Introduction to Servlets & Filters

Servlet Filter: a component that intercepts HTTP requests and responses before they reach a servlet (or controller), used for cross-cutting tasks like logging, compression, or — as Spring Security uses them — authentication and authorization checks.

Spring Security itself is implemented as a chain of filters (the SecurityFilterChain) inserted into the standard servlet filter chain. A custom filter follows the same contract:

```
@Component
public class CustomFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
                         FilterChain chain) throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        System.out.println("Request URL: " + req.getRequestURI());
        chain.doFilter(request, response);
    }
}
```

1.3 Adding Spring Security with Static Credentials

Adding the spring-boot-starter-security dependency immediately secures every endpoint by default with HTTP Basic authentication, using either an auto-generated password (logged on startup) or static credentials defined in properties.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
# application.properties
spring.security.user.name=admin
spring.security.user.password=admin123
spring.security.user.roles=ADMIN
```

HTTP Basic Authentication: an authentication scheme where the client sends credentials (username:password, Base64-encoded) in the Authorization header of every request — simple but only secure over HTTPS, and not suitable for browser-based session UX.

Chapter 2: Changing Default Security Configuration

SecurityFilterChain: a Spring bean that defines an ordered chain of security filters and the authorization rules applied to incoming HTTP requests — the modern (Spring Security 5.7+) replacement for extending WebSecurityConfigurerAdapter.

CSRF (Cross-Site Request Forgery): an attack where a malicious site tricks a logged-in user's browser into submitting an unwanted request to another site where they're authenticated; Spring Security's CSRF protection requires a token to accompany state-changing requests to prevent this.

2.1 Custom Security Configuration

A @Configuration class with a SecurityFilterChain bean replaces Spring Boot's auto-configured defaults, giving full control over which requests require authentication and which authentication mechanisms are enabled.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf().disable()
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults())
            .formLogin(Customizer.withDefaults());
        return http.build();
    }
}
```

permitAll(): an authorization rule that allows unrestricted access to matching requests, with no authentication required.

authenticated(): an authorization rule requiring that the request have a valid, authenticated principal, regardless of role/authority.

2.2 Disabling Form Login and HTTP Basic

Form Login: Spring Security's built-in mechanism for browser-based login via an HTML form, redirecting unauthenticated users to a login page.

For stateless REST APIs (especially those secured with JWT, covered in Chapter 11), both Form Login and HTTP Basic are typically disabled, since the client authenticates once and then presents a bearer token on every request instead.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(authz -> authz
            .anyRequest().authenticated()
        )
        .httpBasic(httpBasic -> httpBasic.disable())
        .formLogin(formLogin -> formLogin.disable());
    return http.build();
}
```

2.3 HTTP Basic Authentication Testing with Postman

The Authorization header value is the Base64 encoding of username:password, prefixed with "Basic ". Tools like Postman or curl can send this directly, or compute it automatically when given plain credentials.

```
# GET request with Basic Auth
curl -X GET http://localhost:8080/api/user \
     -H "Authorization: Basic YWRtaW46YWRtaW4xMjM="
```

Chapter 3: Defining & Managing Users using InMemoryUserDetailsManager

UserDetailsService: the core Spring Security interface with a single method, loadUserByUsername(), responsible for retrieving user data (username, password, authorities) given a username — the bridge between Spring Security and any user store.

UserDetailsManager: an extension of UserDetailsService that adds methods to create, update, and delete users — implemented by InMemoryUserDetailsManager (in-memory, for demos/tests) and JdbcUserDetailsManager (database-backed).

UserDetails: the interface representing a user's core security information: username, password (hashed), enabled/locked/expired flags, and granted authorities.

GrantedAuthority: a single permission granted to an authenticated user, represented as a string (conventionally prefixed ROLE_ for roles, e.g. ROLE_ADMIN); the building block both roles and fine-grained authorities are made of.

3.1 InMemoryUserDetailsManager Configuration

For demos and simple apps, users can be defined directly in configuration rather than a database — useful for prototyping the security flow before wiring up persistence.

```
@Bean
public UserDetailsService users() {
    UserDetails admin = User.builder()
        .username("admin")
        .password("{bcrypt}
$2a$10$dXJ3SW6G7P50lGmMkkmwe.20cQQubK3.HZWzG3YB1tlRy.fqvM/BG")
        .roles("ADMIN")
        .build();

    UserDetails user = User.builder()
        .username("user")
        .password("{bcrypt}
$2a$10$dXJ3SW6G7P50lGmMkkmwe.20cQQubK3.HZWzG3YB1tlRy.fqvM/BG")
        .roles("USER")
        .build();

    return new InMemoryUserDetailsManager(admin, user);
}
```

The {bcrypt} prefix tells Spring Security's DelegatingPasswordEncoder which algorithm was used to hash this particular password, allowing multiple hashing schemes to coexist (e.g. during a migration).

3.2 PasswordEncoder Configuration

PasswordEncoder: an interface for hashing and verifying passwords; encode() hashes a raw password, matches() checks a raw password against a stored hash without ever reversing the hash.

DelegatingPasswordEncoder: a PasswordEncoder implementation that reads the {prefix} on a stored hash to delegate verification to the correct underlying encoder (bcrypt, scrypt, argon2, etc.), supporting algorithm migration over time.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
```

```
}
```

3.3 UserDetailsService and UserDetailsManager Interfaces

In real applications, loadUserByUsername() is implemented to query a database (or any other store) instead of returning in-memory users.

```
// Custom UserDetailsService Implementation
@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));

        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            getAuthorities(user)
        );
    }

    private Collection<? extends GrantedAuthority> getAuthorities(User user) {
        return user.getRoles().stream()
            .map(role -> new SimpleGrantedAuthority("ROLE_" + role.getName()))
            .collect(Collectors.toList());
    }
}
```

Chapter 4: Defining & Managing Users Using a Database

Persistence Layer (for users): the database tables and JPA entities/repositories that durably store user accounts, credentials, and role assignments, replacing the in-memory store from Chapter 3.

4.1 Setting Up MySQL with Docker

```
# Docker command to run MySQL
docker run -d --name mysql-container \
  -e MYSQL_ROOT_PASSWORD=root123 \
  -e MYSQL_DATABASE=eazybank \
  -p 3306:3306 \
  mysql:8.0
```

4.2 Creating Custom Tables for Authentication

A normalized schema separates users, roles, and the many-to-many relationship between them, allowing a user to hold multiple roles.

```
-- Schema for custom tables
CREATE TABLE users (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    enabled BOOLEAN DEFAULT TRUE,
    email VARCHAR(100)
);

CREATE TABLE roles (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE user_roles (
    user_id BIGINT,
    role_id BIGINT,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (role_id) REFERENCES roles(id)
);
```

4.3 JPA Entity Classes

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String username;

    private String password;

    private boolean enabled;

    private String email;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
```

```
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles = new HashSet<>();
}

@Entity
@Table(name = "roles")
public class Role {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

4.4 Registration API

A public endpoint creates new user accounts, hashing the password before storage and assigning a default role.

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody RegisterRequest request) {
        if (userRepository.findByUsername(request.getUsername()).isPresent()) {
            return ResponseEntity.badRequest().body("Username already exists");
        }

        User user = new User();
        user.setUsername(request.getUsername());
        user.setPassword(passwordEncoder.encode(request.getPassword()));
        user.setEnabled(true);

        // Assign default role
        Role userRole = roleRepository.findByName("ROLE_USER")
            .orElseThrow(() -> new RuntimeException("Default role not found"));
        user.getRoles().add(userRole);

        userRepository.save(user);
        return ResponseEntity.ok("User registered successfully");
    }
}
```

Chapter 5: Password Management with PasswordEncoders

Encoding: a reversible transformation (e.g. Base64) used to represent data in a different format — NOT secure for passwords, since it can be trivially reversed.

Encryption: a reversible transformation that requires a secret key to reverse — appropriate for data that must be recovered later (e.g. stored credit card numbers), but not ideal for passwords since the key itself becomes a target.

Hashing: a one-way transformation that cannot be reversed — the correct technique for passwords, since the system only ever needs to verify a password matches, never recover the original value.

5.1 Comparing the Three

```
// Encoding (Not Secure) – reversible, no key needed
String encoded = Base64.getEncoder().encodeToString("password".getBytes());

// Encryption (Reversible) – reversible, requires a key
public class EncryptionDemo {
    public static String encrypt(String text) {
        try {
            Cipher cipher = Cipher.getInstance("AES");
            return
Base64.getEncoder().encodeToString(cipher.doFinal(text.getBytes()));
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}

// Hashing (One-way) – ideal for passwords
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

BCrypt: a deliberately slow, adaptive password-hashing algorithm that incorporates a random salt automatically, making precomputed (rainbow-table) attacks impractical and allowing the computational cost to be tuned as hardware improves.

5.2 BCrypt Password Encoder Implementation

```
@Service
public class AuthService {

    @Autowired
    private PasswordEncoder passwordEncoder;

    public String hashPassword(String rawPassword) {
        return passwordEncoder.encode(rawPassword);
    }

    public boolean verifyPassword(String rawPassword, String encodedPassword) {
        return passwordEncoder.matches(rawPassword, encodedPassword);
    }
}
```

5.3 Registration with BCrypt

```
@PostMapping("/register")
```

```
public ResponseEntity<?> register(@RequestBody RegisterRequest request) {
    User user = new User();
    user.setUsername(request.getUsername());
    user.setPassword(passwordEncoder.encode(request.getPassword()));
    // Password is stored as BCrypt hash
    userRepository.save(user);
    return ResponseEntity.ok("User registered");
}
```

Chapter 6: Understanding Authentication Provider

AuthenticationProvider: the interface responsible for performing the actual authentication logic — given an Authentication request object, it verifies credentials and returns a fully populated, authenticated Authentication object (or throws an exception).

AuthenticationManager: the entry point that delegates authentication requests to one or more configured AuthenticationProvider instances, returning the result from whichever provider successfully authenticates the request.

Authentication: an interface representing a token of authentication — holding the principal (who), credentials (e.g. password, often cleared after authentication), and granted authorities (what they can do).

6.1 Custom AuthenticationProvider

Implementing AuthenticationProvider directly gives full control over the authentication algorithm — useful when the default DaoAuthenticationProvider's behavior isn't sufficient (e.g. custom account-locking logic, multi-factor checks, external identity verification).

```
@Component
public class CustomAuthenticationProvider implements AuthenticationProvider {

    @Autowired
    private UserDetailsService userDetailsService;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @Override
    public Authentication authenticate(Authentication authentication)
            throws AuthenticationException {
        String username = authentication.getName();
        String password = authentication.getCredentials().toString();

        UserDetails userDetails = userDetailsService.loadUserByUsername(username);

        if (!passwordEncoder.matches(password, userDetails.getPassword())) {
            throw new BadCredentialsException("Invalid password");
        }

        return new UsernamePasswordAuthenticationToken(
            userDetails,
            password,
            userDetails.getAuthorities()
        );
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return
UsernamePasswordAuthenticationToken.class.isAssignableFrom(authentication);
    }
}
```

The supports() method tells the AuthenticationManager which Authentication subtypes this provider can handle, allowing multiple providers (e.g. one for username/password, one for API keys) to coexist.

6.2 Environment-Specific Configuration

Security rules often need to differ between environments — relaxed CSRF/HTTPS requirements in development, strict settings in production.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Value("${spring.profiles.active}")
    private String activeProfile;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        if ("dev".equals(activeProfile)) {
            http.csrf().disable();
        } else if ("prod".equals(activeProfile)) {

http.csrf().csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse());
        }

        return http.build();
    }
}
```

Chapter 7: Spring Security Customization for Common Use Cases

7.1 HTTPS Configuration

requiresChannel / requiresSecure: a security rule that forces matching requests to be served over HTTPS, redirecting plain HTTP requests automatically — protects credentials and tokens from being intercepted in transit.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .requiresChannel(channel -> channel
            .anyRequest().requiresSecure()
        );
    return http.build();
}
```

7.2 Custom AuthenticationEntryPoint

AuthenticationEntryPoint: invoked when an unauthenticated user attempts to access a protected resource — by default redirects to a login page, but can be overridden to return a structured JSON 401 response instead, which is essential for REST APIs.

```
@Component
public class CustomAuthenticationEntryPoint implements AuthenticationEntryPoint {

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
                         AuthenticationException authException) throws IOException {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.setContentType("application/json");
        Map<String, Object> errorBody = new HashMap<>();
        errorBody.put("status", 401);
        errorBody.put("error", "Unauthorized");
        errorBody.put("message", authException.getMessage());
        errorBody.put("path", request.getRequestURI());

        response.getWriter().write(new
ObjectMapper().writeValueAsString(errorBody));
    }
}
```

7.3 Custom AccessDeniedHandler

AccessDeniedHandler: invoked when an authenticated user attempts an action they don't have permission for — distinct from AuthenticationEntryPoint, which handles the case of no authentication at all (401 vs 403).

```
@Component
public class CustomAccessDeniedHandler implements AccessDeniedHandler {

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
                       AccessDeniedException accessDeniedException) throws
IOException {
        response.setStatus(HttpServletResponse.SC_FORBIDDEN);
        response.setContentType("application/json");
        Map<String, Object> errorBody = new HashMap<>();
        errorBody.put("status", 403);
```

```
        errorBody.put("error", "Forbidden");
        errorBody.put("message", "You don't have permission to access this
resource");

        response.getWriter().write(new
ObjectMapper().writeValueAsString(errorBody));
    }
}
```

7.4 Session Management

Session Fixation: an attack where an attacker sets a known session ID for a victim before they log in, then hijacks the session once authenticated; mitigated by regenerating the session ID on login (migrateSession()).

SessionCreationPolicy: controls when Spring Security creates an HTTP session: ALWAYS, IF_REQUIRED (default), NEVER, or STATELESS (no session ever created or used — the standard choice for JWT-based APIs, see Chapter 11).

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .sessionManagement(session -> session
            .maximumSessions(1)  // Only one session per user
            .maxSessionsPreventsLogin(false)  // Allow new login, invalidate old
            .expiredUrl("/session-expired")
            .and()
            .sessionFixation().migrateSession()  // Protect against session fixation
            .invalidSessionUrl("/invalid-session")
            .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
        );
    return http.build();
}
```

7.5 Authentication Events Listener

Spring Security publishes events on authentication success/failure, which can be observed for audit logging, alerting on repeated failed logins, or triggering account lockout logic.

```
@Component
public class AuthenticationEventListener {

    @EventListener
    public void onSuccess(AuthenticationSuccessEvent event) {
        Authentication authentication = event.getAuthentication();
        String username = authentication.getName();
        System.out.println("User " + username + " logged in successfully");
    }

    @EventListener
    public void onFailure(AbstractAuthenticationFailureEvent event) {
        AuthenticationException exception = event.getException();
        System.out.println("Authentication failed: " + exception.getMessage());
    }
}
```

7.6 SecurityContext and SecurityContextHolder

SecurityContext: holds the Authentication object for the currently executing thread/request — the place Spring Security stores "who is making this request right now".

SecurityContextHolder: a static accessor for the current SecurityContext, by default backed by a ThreadLocal — this is how any code, anywhere in the call stack, can retrieve the currently authenticated user without it being explicitly passed as a parameter.

```
@RestController
@RequestMapping("/api/user")
public class UserController {

    @GetMapping("/me")
    public ResponseEntity<?> getCurrentUser() {
        Authentication authentication =
SecurityContextHolder.getContext().getAuthentication();
        String username = authentication.getName();
        Collection<? extends GrantedAuthority> authorities =
authentication.getAuthorities();

        Map<String, Object> userInfo = new HashMap<>();
        userInfo.put("username", username);
        userInfo.put("authorities", authorities.stream()
            .map(GrantedAuthority::getAuthority)
            .collect(Collectors.toList()));

        return ResponseEntity.ok(userInfo);
    }
}
```

Chapter 8: Understanding CORS & CSRF

CORS (Cross-Origin Resource Sharing): a browser security mechanism that blocks JavaScript on one origin (domain+port+protocol) from making requests to a different origin, unless the server explicitly allows it via response headers — necessary whenever a frontend (e.g. Angular on :4200) calls a backend on a different origin (e.g. :8080).

CSRF was defined in Chapter 2 — it protects against forged requests from a browser that already has a valid session; CORS, in contrast, controls which origins are even allowed to make cross-origin requests in the first place. The two are often configured together but solve different problems.

8.1 CORS Configuration

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        .csrf(csrf -> csrf.disable());
    return http.build();
}

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();
    configuration.setAllowedOrigins(Arrays.asList("http://localhost:4200"));
    configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE"));
    configuration.setAllowedHeaders(Arrays.asList("Authorization", "Content-Type"));
    configuration.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);
    return source;
}
```

8.2 CSRF Protection

CsrfToken: a unique, unpredictable value generated per session (or per request) that must be included in state-changing requests (POST/PUT/DELETE); since an attacker's forged cross-site request can't know this token, the request is rejected.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf
            .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
            .ignoringRequestMatchers("/api/auth/**")  // Ignore for public APIs
        );
    return http.build();
}
```

Note: CSRF protection is primarily relevant to cookie/session-based authentication. Stateless JWT-based APIs (Chapter 11) typically disable CSRF entirely, since there's no ambient session cookie for a forged request to ride along on.

8.3 CSRF Token Implementation

```
@RestController
@RequestMapping("/api/csrf")
public class CsrfController {
```

```
    @GetMapping("/token")
    public ResponseEntity<?> getCsrfToken(HttpServletRequest request) {
        CsrfToken csrfToken = (CsrfToken)
request.getAttribute(CsrfToken.class.getName());
        if (csrfToken != null) {
            Map<String, String> token = new HashMap<>();
            token.put("header", csrfToken.getHeaderName());
            token.put("parameter", csrfToken.getParameterName());
            token.put("token", csrfToken.getToken());
            return ResponseEntity.ok(token);
        }
        return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
    }
}
```

Chapter 9: Implementing Authorization using Authorities & Roles

Role: a coarse-grained grouping of permissions assigned to a user (e.g. ROLE_ADMIN), typically representing a job function; in Spring Security, roles are just GrantedAuthority strings with a ROLE_ prefix by convention.

Authority: a fine-grained, individual permission (e.g. READ, MANAGE_USERS); a single role is often composed of multiple authorities, giving more granular control than roles alone.

9.1 Authorities vs Roles

```
// Authorities are more granular permissions
public enum Authority {
    READ,
    WRITE,
    DELETE,
    MANAGE_USERS
}

// Roles are collections of authorities
public enum Role {
    ROLE_USER,
    ROLE_ADMIN,
    ROLE_MANAGER
}
```

9.2 Creating Authorities Table

```
CREATE TABLE authorities (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL,
    authority VARCHAR(50) NOT NULL,
    FOREIGN KEY (username) REFERENCES users(username)
);

CREATE TABLE user_authorities (
    user_id BIGINT,
    authority_id BIGINT,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (authority_id) REFERENCES authorities(id)
);
```

9.3 Role-Based Authorization

hasRole() / hasAuthority(): authorization expressions used in request matchers or method security: hasRole("ADMIN") checks for the authority ROLE_ADMIN (prefix added automatically); hasAuthority("MANAGE_USERS") checks for an exact authority string with no prefix assumed.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(authz -> authz
                // Authority-based
                .requestMatchers("/api/admin/**").hasAuthority("MANAGE_USERS")
                .requestMatchers("/api/read/**").hasAuthority("READ")
```

```
                // Role-based
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")
                .requestMatchers("/api/public/**").permitAll()
                .anyRequest().authenticated()
            );
        return http.build();
    }
}
```

9.4 Method-Level Security

@PreAuthorize: a method-level annotation that evaluates a SpEL security expression before the method executes, throwing an AccessDeniedException (mapped to a 403) if it evaluates false — covered in depth in Chapter 12.

```
@RestController
@RequestMapping("/api/admin")
@PreAuthorize("hasRole('ADMIN')")
public class AdminController {

    @GetMapping("/users")
    @PreAuthorize("hasAuthority('MANAGE_USERS')")
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }

    @DeleteMapping("/users/{id}")
    @PreAuthorize("hasRole('ADMIN') and hasAuthority('DELETE')")
    public ResponseEntity<?> deleteUser(@PathVariable Long id) {
        userService.deleteUser(id);
        return ResponseEntity.ok().build();
    }
}
```

Chapter 10: Custom Filters in Spring Security

OncePerRequestFilter: a Spring-provided base class guaranteeing a filter's doFilterInternal() executes exactly once per request, even in environments with internal request forwarding/dispatching — the standard base class for custom security filters.

Filter Chain Position: where a custom filter is inserted relative to Spring Security's built-in filters (addFilterBefore/After/At), which determines whether it runs before authentication has happened, after, or replaces a built-in filter entirely.

10.1 Creating Custom Filters

```
@Component
public class RequestLoggingFilter extends OncePerRequestFilter {

    private static final Logger logger =
LoggerFactory.getLogger(RequestLoggingFilter.class);

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        logger.info("Request: {} {}", request.getMethod(), request.getRequestURI());

        long startTime = System.currentTimeMillis();
        filterChain.doFilter(request, response);
        long duration = System.currentTimeMillis() - startTime;

        logger.info("Response status: {}, Duration: {}ms",
                    response.getStatus(), duration);
    }
}
```

10.2 Adding Filters to Security Chain

Method	Effect
addFilterBefore(filter, X.class)	Inserts the filter immediately before filter X in the chain
addFilterAfter(filter, X.class)	Inserts the filter immediately after filter X in the chain
addFilterAt(filter, X.class)	Inserts the filter at the same position as X (does not replace X — multiple filters can share a position)

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .addFilterBefore(requestLoggingFilter,
UsernamePasswordAuthenticationFilter.class)
        .addFilterAfter(customFilter, BasicAuthenticationFilter.class)
        .addFilterAt(customFilterAt, BasicAuthenticationFilter.class);
    return http.build();
}
```

10.3 Custom Authentication Filter

A JWT authentication filter is the most common custom filter in modern Spring Security apps — it runs before the standard username/password filter, extracts and validates a bearer token, and

populates the SecurityContext if valid, allowing the rest of the chain (including authorization checks) to proceed normally.

```
@Component
public class CustomAuthenticationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtil jwtUtil;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);
            try {
                String username = jwtUtil.extractUsername(token);
                if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
                    UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

                    if (jwtUtil.validateToken(token, userDetails)) {
                        UsernamePasswordAuthenticationToken authToken =
                            new UsernamePasswordAuthenticationToken(userDetails,
null,

userDetails.getAuthorities());

SecurityContextHolder.getContext().setAuthentication(authToken);
                    }
                }
            } catch (Exception e) {
                logger.error("Invalid token: {}", e.getMessage());
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

Chapter 11: Token-based Authentication using JWT

JWT (JSON Web Token): a compact, URL-safe, self-contained token format consisting of three Base64-encoded parts — header.payload.signature — used to represent claims (e.g. username, roles, expiry) that the server can verify without needing to store session state.

Stateless Authentication: an approach where the server does not store any session data between requests; instead, the client presents a signed token (the JWT) on every request, and the server validates its signature and expiry to trust its contents.

Claims: the data encoded inside a JWT's payload — standard claims like sub (subject/username), iat (issued at), exp (expiration), plus any custom claims (e.g. roles) the application adds.

Signing Key: the secret (for HMAC algorithms like HS256) or private key (for RSA/EC algorithms) used to generate the JWT's signature, and the corresponding secret/public key used to verify it — anyone possessing the signing key could forge valid tokens, so it must be protected.

11.1 JWT Utility Class

A utility class centralizes token generation, parsing, and validation logic.

```
@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private Long expiration;

    public String generateToken(String username) {
        Map<String, Object> claims = new HashMap<>();
        return createToken(claims, username);
    }

    private String createToken(Map<String, Object> claims, String subject) {
        return Jwts.builder()
            .setClaims(claims)
            .setSubject(subject)
            .setIssuedAt(new Date(System.currentTimeMillis()))
            .setExpiration(new Date(System.currentTimeMillis() + expiration))
            .signWith(getSignKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    private Key getSignKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secret);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    public Boolean validateToken(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername()) && !
isTokenExpired(token));
    }

    public String extractUsername(String token) {
        return extractAllClaims(token).getSubject();
```

```
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(getSignKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
    }
}
```

HS256: HMAC using SHA-256 — a symmetric signing algorithm where the same secret key both signs and verifies the token; simple to set up but requires every verifying service to share the secret. (RS256, using asymmetric RSA keys, is the common alternative when verification needs to happen in services that shouldn't hold the signing secret.)

11.2 Authentication Controller

The login endpoint authenticates credentials via the AuthenticationManager (Chapter 6) and, on success, issues a JWT instead of creating a session.

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private JwtUtil jwtUtil;

    @Autowired
    private UserDetailsService userDetailsService;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest request) {
        try {
            Authentication authentication = authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(request.getUsername(),
                                                        request.getPassword())
            );

            String token = jwtUtil.generateToken(request.getUsername());

            Map<String, Object> response = new HashMap<>();
            response.put("token", token);
            response.put("username", request.getUsername());
            response.put("expiresIn", jwtUtil.getExpiration());

            return ResponseEntity.ok(response);
        } catch (BadCredentialsException e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                .body("Invalid username or password");
        }
    }
}
```

11.3 JWT Authentication Configuration

Putting it together: the filter chain disables CSRF (no session cookies to forge), sets the session policy to STATELESS, and inserts the JWT filter (Chapter 10.3) before the standard username/password filter so every request is checked for a valid bearer token.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration authConfig) throws Exception {
        return authConfig.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/api/auth/**").permitAll()
                .anyRequest().authenticated()
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .authenticationProvider(authenticationProvider())
            .addFilterBefore(jwtAuthenticationFilter,
                             UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

11.4 JWT Request/Response Flow

Step	What Happens
1. Login	Client POSTs username/password to /api/auth/login; server validates via AuthenticationManager
2. Token issued	Server generates a signed JWT containing the username and expiry, returns it in the response body
3. Client stores token	Client keeps the token (e.g. in memory or local storage) and includes it as Authorization: Bearer <token> on subsequent requests
4. Token validated per request	JwtAuthenticationFilter extracts the token, verifies the signature and expiry, and populates SecurityContextHolder if valid
5. Authorization checks proceed	Once authenticated, normal hasRole()/hasAuthority()/@PreAuthorize checks (Chapters 9, 12) apply as usual

Chapter 12: Method Level Security

Method-Level Security: authorization rules applied directly on service/controller methods (rather than only at the URL/request-matcher level), enforced via AOP proxies that intercept the method call and evaluate a security expression before allowing execution.

@EnableMethodSecurity: the annotation that activates method-level security annotations (@PreAuthorize, @PostAuthorize, @PreFilter, @PostFilter) across the application — the modern replacement for the older @EnableGlobalMethodSecurity.

12.1 Enabling Method Security

```
@Configuration
@EnableMethodSecurity
public class MethodSecurityConfig {
    // Configuration for method-level security
}
```

12.2 PreAuthorize and PostAuthorize

@PreAuthorize: evaluates its SpEL expression BEFORE the method runs — if it evaluates to false, the method body never executes at all.

@PostAuthorize: evaluates its SpEL expression AFTER the method runs, with access to the method's return value via returnObject — useful when the authorization decision depends on data only available after the operation (e.g. checking the owner of the object that was just fetched).

```
@RestController
@RequestMapping("/api/accounts")
public class AccountController {

    @GetMapping("/{id}")
    @PreAuthorize("hasRole('USER') and #id == authentication.principal.id")
    public Account getAccount(@PathVariable Long id) {
        return accountService.getAccount(id);
    }

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Account> createAccount(@RequestBody Account account) {
        return ResponseEntity.ok(accountService.createAccount(account));
    }

    @GetMapping("/secure")
    @PreAuthorize("hasAuthority('READ') and hasRole('ADMIN')")
    public String secureEndpoint() {
        return "Secure data";
    }

    @PostAuthorize("returnObject.username == authentication.principal.username")
    @GetMapping("/profile")
    public UserProfile getProfile() {
        return userService.getCurrentUserProfile();
    }
}
```

12.3 PreFilter and PostFilter

@PreFilter: filters a method's collection-type ARGUMENT before the method executes, removing elements that don't satisfy the expression (filterObject refers to each element being evaluated).

@PostFilter: filters a method's collection-type RETURN VALUE after execution, removing elements the current user shouldn't see — useful for enforcing row-level/ownership-based access control without changing the underlying query.

```
@Service
public class DataService {

    @PreFilter("filterObject.owner == authentication.principal.username")
    public List<Document> processDocuments(List<Document> documents) {
        return documents.stream()
            .map(this::process)
            .collect(Collectors.toList());
    }

    @PostFilter("filterObject.owner == authentication.principal.username")
    public List<Document> getDocuments() {
        List<Document> allDocuments = documentRepository.findAll();
        return allDocuments;
    }
}
```

Caution: @PostFilter filters results in application memory after they've already been fetched from the database — for large datasets, filtering at the query level (e.g. a WHERE owner = :username clause) is far more efficient.

12.4 Security with SpEL Expressions

Custom Security Bean in SpEL: SpEL expressions can call out to any Spring bean's methods (using @beanName.method(...)) for complex authorization logic that doesn't fit neatly into hasRole()/hasAuthority() alone.

```
@Component
public class SecurityEvaluator {

    @PreAuthorize("@securityEvaluator.isAllowed(authentication, #userId)")
    public void performAction(Long userId) {
        // Custom security evaluation
    }

    public boolean isAllowed(Authentication authentication, Long userId) {
        boolean isAdmin = authentication.getAuthorities().stream()
            .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));
        boolean isOwner = authentication.getName().equals(userId.toString());
        return isAdmin || isOwner;
    }
}
```

Chapter 13: Deep Dive of OAuth2 & OpenID Connect

OAuth2: an authorization framework that lets a third-party application obtain limited access to a user's resources on another service, without exposing the user's credentials to that third party — built around issuing scoped, time-limited access tokens.

OpenID Connect (OIDC): an identity layer built on top of OAuth2 that adds authentication (verifying who the user is, via an ID Token) on top of OAuth2's authorization capabilities — this is what powers "Sign in with Google/GitHub".

Grant Type: the OAuth2 flow used to obtain an access token, chosen based on the type of client and trust level (e.g. authorization_code for web apps with a backend, client_credentials for machine-to-machine, refresh_token to renew an expired token without re-authenticating).

Scope: a string identifying a specific permission being requested (e.g. read, write, openid, profile) — the access token is limited to only the scopes that were granted.

13.1 OAuth2 Terminology

```
public class OAuth2Constants {
    // Grant Types
    public static final String AUTHORIZATION_CODE = "authorization_code";
    public static final String CLIENT_CREDENTIALS = "client_credentials";
    public static final String PASSWORD = "password";
    public static final String REFRESH_TOKEN = "refresh_token";

    // Response Types
    public static final String CODE = "code";
    public static final String TOKEN = "token";

    // Scope
    public static final String READ = "read";
    public static final String WRITE = "write";
    public static final String OPENID = "openid";
}
```

13.2 Key OAuth2 Roles

Role	Definition
Resource Owner	The end user who owns the data and grants permission to access it
Client	The application requesting access on the user's behalf (e.g. the eShop frontend)
Authorization Server	Issues access tokens after authenticating the resource owner and obtaining consent (e.g. Google, Keycloak, or a custom Spring Authorization Server)
Resource Server	Hosts the protected resources and validates access tokens on incoming requests (e.g. the eShop backend API)

13.3 OAuth2 Client Configuration

oauth2Login: Spring Security's support for the OAuth2/OIDC "Login" flow — the application itself becomes an OAuth2 client, redirecting users to an external Authorization Server to log in.

```
@Configuration
```

```
@EnableWebSecurity
public class OAuth2Config {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .oauth2Login(oauth2 -> oauth2
                .clientRegistrationRepository(clientRegistrationRepository())
                .authorizedClientService(authorizedClientService())
                .loginPage("/login")
            )
            .oauth2Client(Customizer.withDefaults())
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt ->
jwt.jwtAuthenticationConverter(jwtAuthenticationConverter()))
            );

        return http.build();
    }

    @Bean
    public ClientRegistrationRepository clientRegistrationRepository() {
        ClientRegistration google = ClientRegistration.withRegistrationId("google")
            .clientId("your-google-client-id")
            .clientSecret("your-google-client-secret")
            .scope("openid", "profile", "email")
            .authorizationUri("https://accounts.google.com/o/oauth2/v2/auth")
            .tokenUri("https://www.googleapis.com/oauth2/v4/token")
            .userInfoUri("https://www.googleapis.com/oauth2/v3/userinfo")
            .userNameAttributeName("sub")
            .jwkSetUri("https://www.googleapis.com/oauth2/v3/certs")
            .clientName("Google")
            .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
            .redirectUri("{baseUrl}/{action}/oauth2/code/{registrationId}")
            .build();

        return new InMemoryClientRegistrationRepository(google);
    }
}
```

13.4 OAuth2 Resource Server

Resource Server: in Spring Security terms, oauth2ResourceServer() configures the app to validate incoming bearer tokens (typically JWTs) issued by a separate Authorization Server, rather than issuing tokens itself.

```
@Configuration
@EnableWebSecurity
public class ResourceServerConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt -> jwt.decoder(jwtDecoder()))
                .authenticationEntryPoint(authenticationEntryPoint())
                .accessDeniedHandler(accessDeniedHandler())
            )
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
```

```
                .anyRequest().authenticated()
            );

        return http.build();
    }

    @Bean
    public JwtDecoder jwtDecoder() {
        return NimbusJwtDecoder.withIssuerLocation("https://your-auth-server")
            .build();
    }
}
```

Chapter 14: Implementing OAuth2 with Spring Security & Social Logins

Social Login: using a third-party identity provider (Google, GitHub, Facebook) as the Authorization Server, so users authenticate with an account they already have instead of creating new credentials for your app.

14.1 Social Login Configuration

```
# application.yml
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: your-google-client-id
            client-secret: your-google-client-secret
          github:
            client-id: your-github-client-id
            client-secret: your-github-client-secret
          facebook:
            client-id: your-facebook-client-id
            client-secret: your-facebook-client-secret
```

14.2 OAuth2 User Service

OAuth2UserService: the extension point used to load (and optionally enrich/persist) user information returned by the OAuth2 provider after a successful login — commonly used to create or update a local user record on first login.

```
@Service
public class CustomOAuth2UserService extends DefaultOAuth2UserService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public OAuth2User loadUser(OAuth2UserRequest userRequest) throws
OAuth2AuthenticationException {
        OAuth2User oAuth2User = super.loadUser(userRequest);

        String provider = userRequest.getClientRegistration().getRegistrationId();
        String providerId = oAuth2User.getAttribute("sub");
        String email = oAuth2User.getAttribute("email");
        String name = oAuth2User.getAttribute("name");

        User user = userRepository.findByProviderAndProviderId(provider, providerId)
            .orElseGet(() -> {
                User newUser = new User();
                newUser.setProvider(provider);
                newUser.setProviderId(providerId);
                newUser.setEmail(email);
                newUser.setName(name);
                newUser.setEnabled(true);
                newUser.setRole(roleRepository.findByName("ROLE_USER")
                    .orElseThrow(() -> new RuntimeException("Default role not
found")));
                return userRepository.save(newUser);
            });
```

```
        return new DefaultOAuth2User(
            List.of(new SimpleGrantedAuthority("ROLE_" + user.getRole().getName())),
            oAuth2User.getAttributes(),
            "email"
        );
    }
}
```

14.3 OAuth2 Success Handler

AuthenticationSuccessHandler: invoked after a successful OAuth2 login, used here to issue the application's own JWT (Chapter 11) and redirect the browser back to the frontend with that token — bridging an external social login into the app's own stateless authentication scheme.

```
@Component
public class OAuth2AuthenticationSuccessHandler implements
AuthenticationSuccessHandler {

    @Autowired
    private JwtUtil jwtUtil;

    @Autowired
    private UserService userService;

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request,
                                        HttpServletResponse response,
                                        Authentication authentication) throws
IOException {
        OAuth2User oAuth2User = (OAuth2User) authentication.getPrincipal();
        String email = oAuth2User.getAttribute("email");

        User user = userService.findByEmail(email);
        String jwtToken = jwtUtil.generateToken(user.getUsername());

        String redirectUrl = "http://localhost:4200/oauth2/redirect?token=" +
jwtToken;
        response.sendRedirect(redirectUrl);
    }
}
```

Chapter 15: Securing EazyBank with OAuth2 and KeyCloak

Keycloak: an open-source Identity and Access Management (IAM) solution that acts as a full OAuth2/OIDC Authorization Server — handling login UI, user storage, MFA, social login federation, and token issuance, so applications don't need to build this themselves.

Realm: Keycloak's term for an isolated tenant/namespace containing its own users, roles, and client applications — e.g. an "eazybank" realm separate from other applications on the same Keycloak instance.

Opaque Token: an access token whose contents are NOT self-describing/parseable by the resource server (unlike a JWT) — the resource server must call back to the Authorization Server's introspection endpoint to validate it and learn its associated claims.

15.1 KeyCloak Configuration

```
# application.yml for KeyCloak resource server
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8080/realms/eazybank
          jwk-set-uri: http://localhost:8080/realms/eazybank/protocol/openid-
connect/certs
```

15.2 KeyCloak Resource Server Configuration

```
@Configuration
@EnableWebSecurity
public class ResourceServerConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt -> jwt.decoder(jwtDecoder()))
            )
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            );

        return http.build();
    }

    @Bean
    public JwtDecoder jwtDecoder() {
        return NimbusJwtDecoder.withIssuerLocation("http://localhost:8080/realms/
eazybank")
            .build();
    }
}
```

15.3 KeyCloak Integration with Angular

A frontend single-page app integrates with Keycloak directly (via the keycloak-js adapter) to handle the login redirect and token acquisition, then attaches the resulting token to API calls to the Spring Boot resource server above.

```
// Angular KeyCloak Service
import { Injectable } from '@angular/core';
import Keycloak from 'keycloak-js';

@Injectable({
  providedIn: 'root'
})
export class KeycloakService {
  private keycloak: Keycloak.KeycloakInstance;

  constructor() {
    this.keycloak = new Keycloak({
      url: 'http://localhost:8080',
      realm: 'eazybank',
      clientId: 'eazybank-angular'
    });
  }

  async init() {
    const authenticated = await this.keycloak.init({
      onLoad: 'login-required',
      checkLoginIframe: false
    });
    return authenticated;
  }

  getToken(): string {
    return this.keycloak.token;
  }

  logout() {
    this.keycloak.logout();
  }
}
```

15.4 Opaque Token Configuration

```
@Configuration
@EnableWebSecurity
public class OpaqueTokenConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .oauth2ResourceServer(oauth2 -> oauth2
                .opaqueToken(opaqueToken -> opaqueToken
                    .introspector(opaqueTokenIntrospector())
                )
            );

        return http.build();
    }

    @Bean
    public OpaqueTokenIntrospector opaqueTokenIntrospector() {
```

```
        return new NimbusOpaqueTokenIntrospector(
            "http://localhost:8080/realms/eazybank/protocol/openid-connect/token/
introspect",
            "resource-server-client-id",
            "resource-server-client-secret"
        );
    }
}
```

JWT vs Opaque Token trade-off: JWTs can be validated locally (just check the signature against a public key) with no network call, making them fast and suitable for high-throughput resource servers; opaque tokens require a network round-trip to introspect on every request, but allow the Authorization Server to revoke a token instantly (JWTs remain valid until expiry unless additional revocation infrastructure is added).

Chapter 16: OAuth2 Auth Server using Spring Authorization Server

Spring Authorization Server: a Spring project that lets you build your own fully-featured OAuth2/OIDC Authorization Server (rather than relying on Keycloak or a third party), useful when you need full control over the token issuance process, custom claims, or tight integration with existing Spring infrastructure.

RegisteredClient: represents an OAuth2 client application registered with the Authorization Server — its allowed grant types, redirect URIs, scopes, and authentication method.

16.1 Spring Authorization Server Setup

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-authorization-server</artifactId>
</dependency>
```

16.2 Authorization Server Configuration

```
@Configuration
@EnableWebSecurity
public class AuthServerConfig {

    @Bean
    public SecurityFilterChain authorizationServerSecurityFilterChain(
            HttpSecurity http) throws Exception {

        OAuth2AuthorizationServerConfiguration.applyDefaultSecurity(http);

        http
            .getConfigurer(OAuth2AuthorizationServerConfigurer.class)
            .oidc(Customizer.withDefaults());

        http
            .exceptionHandling(exceptions -> exceptions
                .authenticationEntryPoint(
                    new LoginUrlAuthenticationEntryPoint("/login"))
            );

        return http.build();
    }

    @Bean
    public RegisteredClientRepository registeredClientRepository() {
        RegisteredClient registeredClient =
RegisteredClient.withId(UUID.randomUUID().toString())
            .clientId("client-id")
            .clientSecret("{bcrypt}" + new BCryptPasswordEncoder().encode("client-
secret"))
            .clientAuthenticationMethod(ClientAuthenticationMethod.CLIENT_SECRET_BAS
IC)
            .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
            .authorizationGrantType(AuthorizationGrantType.REFRESH_TOKEN)
            .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
            .redirectUri("http://localhost:8080/login/oauth2/code/client-id")
            .scope("openid")
            .scope("profile")
            .scope("read")
            .scope("write")
```

```
            .clientSettings(ClientSettings.builder()
                .requireAuthorizationConsent(true)
                .build())
            .build();

        return new InMemoryRegisteredClientRepository(registeredClient);
    }
}
```

16.3 OAuth2 Token Customization

Custom claims (e.g. application-specific roles) can be added to issued tokens by customizing the JwtGenerator — useful when downstream resource servers need more context than the standard OIDC claims provide.

```
@Configuration
public class OAuth2TokenConfig {

    @Bean
    public OAuth2TokenGenerator<?> tokenGenerator(
            JwtGenerator jwtGenerator,
            OAuth2AccessTokenGenerator accessTokenGenerator,
            OAuth2RefreshTokenGenerator refreshTokenGenerator) {

        return new DelegatingOAuth2TokenGenerator(
            jwtGenerator,
            accessTokenGenerator,
            refreshTokenGenerator
        );
    }

    @Bean
    public JwtGenerator jwtGenerator(JwtEncoder jwtEncoder) {
        return new JwtGenerator(jwtEncoder) {
            @Override
            public OAuth2Token generate(OAuth2TokenContext context) {
                Jwt jwt = (Jwt) super.generate(context);
                Map<String, Object> claims = new HashMap<>(jwt.getClaims());
                claims.put("custom-claim", "custom-value");
                claims.put("user-role", context.getPrincipal().getAuthorities());

                return new Jwt(
                    jwt.getTokenValue(),
                    jwt.getIssuedAt(),
                    jwt.getExpiresAt(),
                    jwt.getHeaders(),
                    claims
                );
            }
        };
    }
}
```

16.4 Database-Backed Client Registration

For production, registered clients are persisted in a database rather than declared in-memory, allowing clients to be added/updated without redeploying the Authorization Server.

```
@Entity
@Table(name = "oauth2_clients")
public class OAuth2Client {
```

```
    @Id
    private String id;

    private String clientId;
    private String clientSecret;
    private String clientAuthenticationMethods;
    private String authorizationGrantTypes;
    private String redirectUris;
    private String scopes;
    private boolean requireAuthorizationConsent;
    private boolean requireProofKey;
}

@Component
public class JpaRegisteredClientRepository implements RegisteredClientRepository {

    @Autowired
    private OAuth2ClientRepository clientRepository;

    @Override
    public void save(RegisteredClient registeredClient) {
        // Convert RegisteredClient to OAuth2Client entity and save
    }

    @Override
    public RegisteredClient findById(String id) {
        return clientRepository.findById(id)
            .map(this::toRegisteredClient)
            .orElse(null);
    }

    @Override
    public RegisteredClient findByClientId(String clientId) {
        return clientRepository.findByClientId(clientId)
            .map(this::toRegisteredClient)
            .orElse(null);
    }

    private RegisteredClient toRegisteredClient(OAuth2Client client) {
        return RegisteredClient.withId(client.getId())
            .clientId(client.getClientId())
            .clientSecret(client.getClientSecret())
            .clientAuthenticationMethod(parseAuthMethods(client.getClientAuthenticat
ionMethods()))
            .authorizationGrantType(parseGrantTypes(client.getAuthorizationGrantType
s()))
            .redirectUri(parseRedirectUris(client.getRedirectUris()))
            .scope(parseScopes(client.getScopes()))
            .clientSettings(ClientSettings.builder()
                .requireAuthorizationConsent(client.isRequireAuthorizationConsent())
                .build())
            .build();
    }
}
```

Appendix A: Complete Application Structure

```
src/
├── main/
│   ├── java/
│   │   └── com/eazybank/
│   │       ├── SpringSecurityApplication.java
│   │       ├── config/
│   │       │   ├── SecurityConfig.java
│   │       │   ├── JwtConfig.java
│   │       │   └── OAuth2Config.java
│   │       ├── controller/
│   │       │   ├── AuthController.java
│   │       │   ├── UserController.java
│   │       │   └── AdminController.java
│   │       ├── service/
│   │       │   ├── UserService.java
│   │       │   ├── CustomUserDetailsService.java
│   │       │   └── JwtService.java
│   │       ├── filter/
│   │       │   ├── JwtAuthenticationFilter.java
│   │       │   └── RequestLoggingFilter.java
│   │       ├── handler/
│   │       │   ├── CustomAuthenticationEntryPoint.java
│   │       │   └── CustomAccessDeniedHandler.java
│   │       ├── entity/
│   │       │   ├── User.java
│   │       │   ├── Role.java
│   │       │   └── Authority.java
│   │       ├── repository/
│   │       │   ├── UserRepository.java
│   │       │   └── RoleRepository.java
│   │       └── dto/
│   │           ├── LoginRequest.java
│   │           └── RegisterRequest.java
│   └── resources/
│       ├── application.yml
│       └── db/
│           └── schema.sql
└── test/
    └── java/
        └── com/eazybank/
            └── SecurityTests.java
```

Appendix B: Docker Compose for Development

A full local stack — MySQL for persistence, Keycloak for OAuth2/OIDC, Redis for caching, and the application itself — runnable with a single command.

```
version: '3.8'

services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: eazybank
    ports:
      - "3306:3306"
    volumes:
```

```
      - mysql_data:/var/lib/mysql

  keycloak:
    image: quay.io/keycloak/keycloak:latest
    environment:
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin123
    ports:
      - "8080:8080"
    command: start-dev

  redis:
    image: redis:alpine
    ports:
      - "6379:6379"

  app:
    build: .
    ports:
      - "8081:8080"
    depends_on:
      - mysql
      - keycloak
      - redis
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/eazybank
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: root123
      SPRING_REDIS_HOST: redis

volumes:
  mysql_data:
```

Appendix C: Key Terms Glossary (Quick Reference)

Term	One-Line Definition
Authentication	Verifying who a user is
Authorization	Verifying what an authenticated user can do
SecurityFilterChain	Ordered chain of filters defining security rules for incoming requests
UserDetailsService	Loads user data (username, password, authorities) by username
PasswordEncoder	Hashes and verifies passwords one-way (e.g. BCrypt)
AuthenticationProvider	Performs the actual credential-checking logic
SecurityContextHolder	Thread-local holder for the current request's Authentication
CSRF	Protection against forged state-changing requests from a user's own browser/session
CORS	Browser policy controlling which origins may call your API from JavaScript

JWT	Self-contained, signed token carrying claims, enabling stateless authentication
OAuth2	Framework for delegated, scoped authorization via access tokens
OIDC	Identity layer on OAuth2 adding authentication (ID Tokens)
Keycloak	Open-source Identity and Access Management server (Authorization Server)
@PreAuthorize / @PostAuthorize	Method-level security checks before/after method execution